

Text Indexing in Compressed Space

Tomasz Kociumaka



MAX PLANCK INSTITUTE
FOR INFORMATICS

SIC Saarland Informatics
Campus

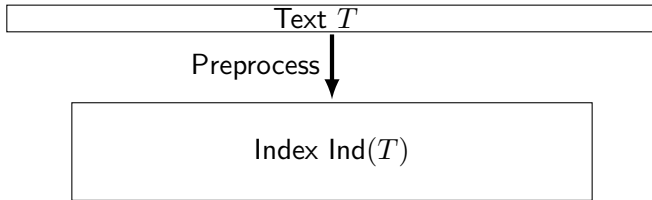
NII Theory Seminar
May 22nd, 2026

Based on joint works with Dominik Kempa, Gonzalo Navarro, Francisco Olivares, and Nicola Prezza.

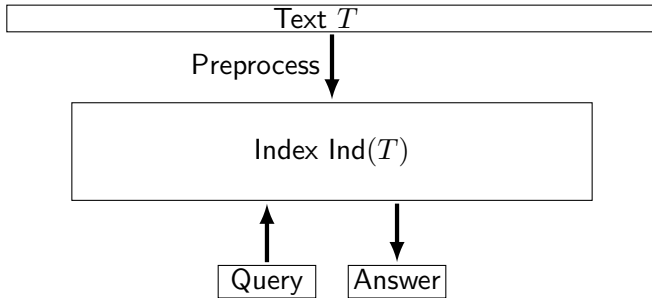
Text Indexing

Text T

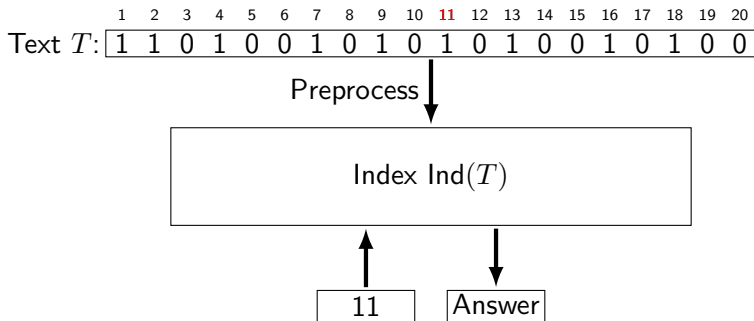
Text Indexing



Text Indexing



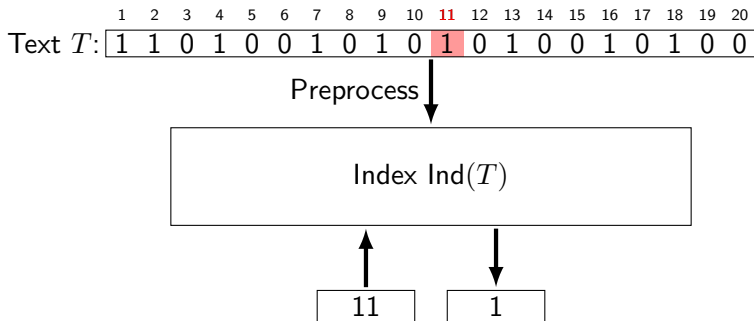
Text Indexing



Basic queries:

Random Access: Retrieve the character of T at a given position.

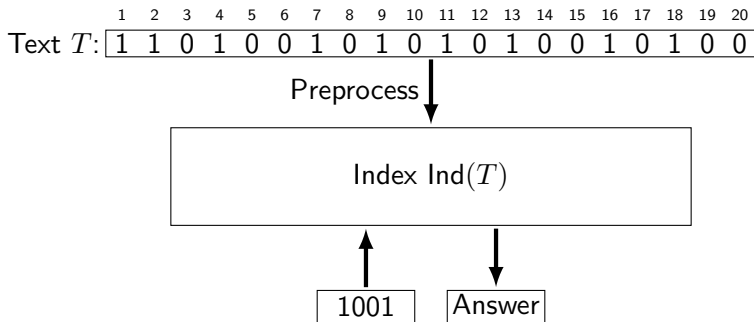
Text Indexing



Basic queries:

Random Access: Retrieve the character of T at a given position.

Text Indexing

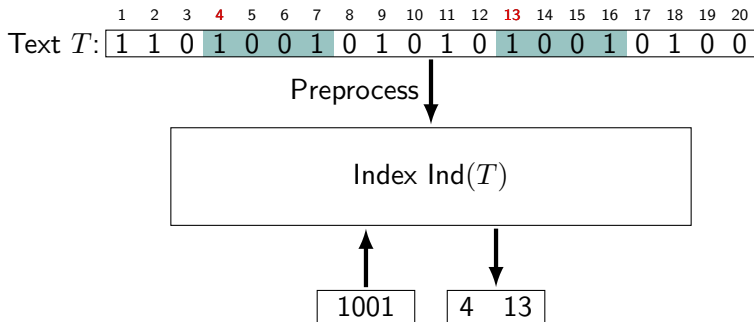


Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Text Indexing

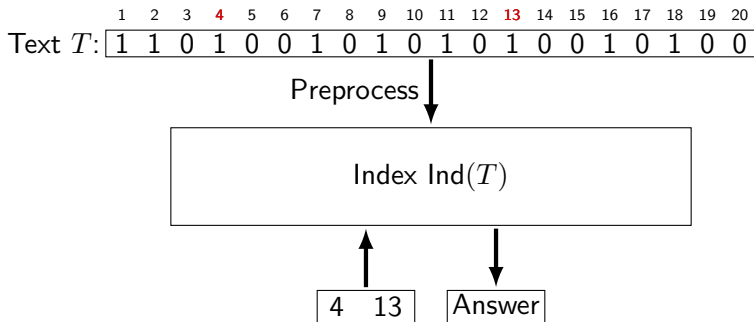


Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Text Indexing



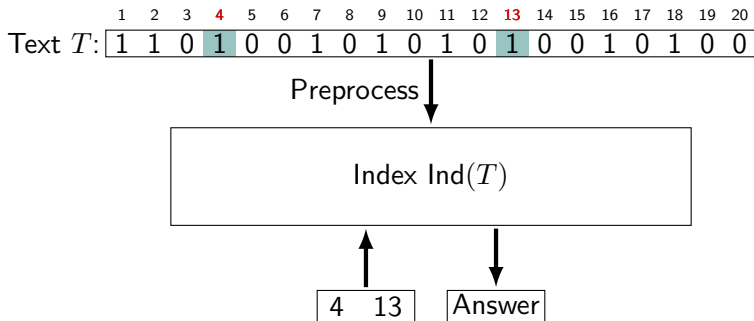
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



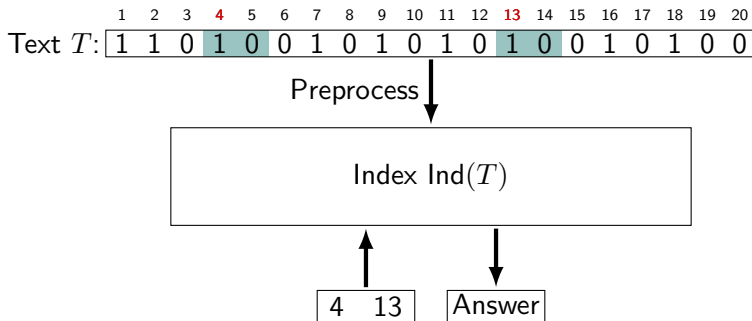
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



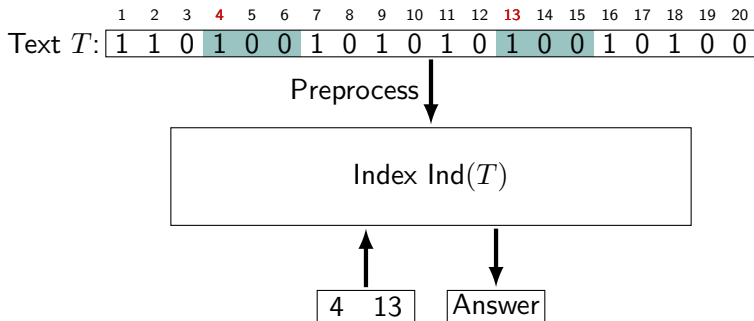
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



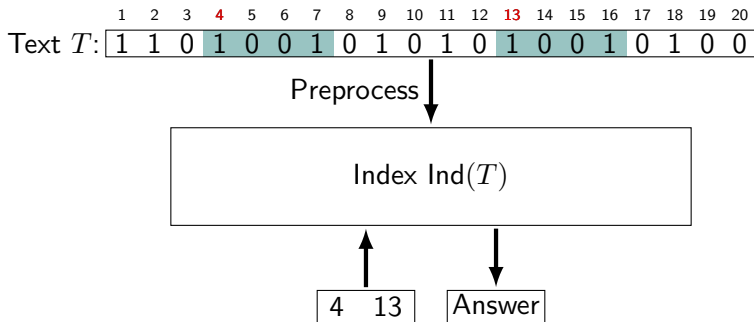
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



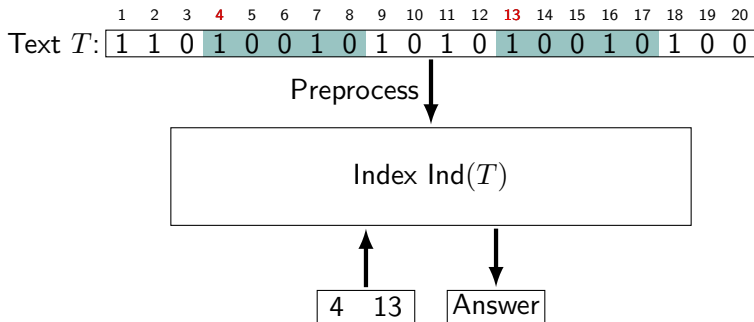
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



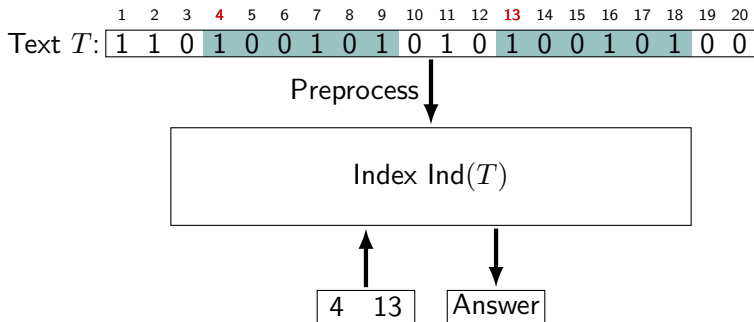
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



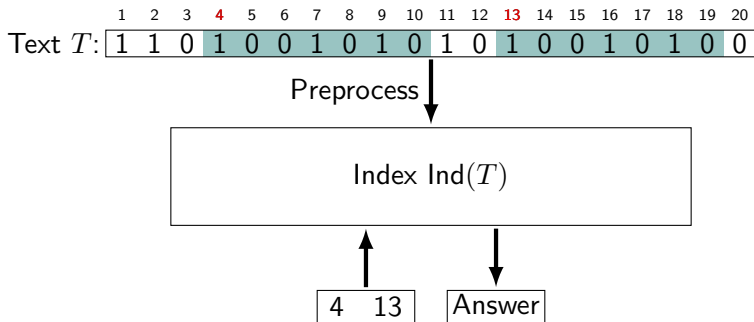
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



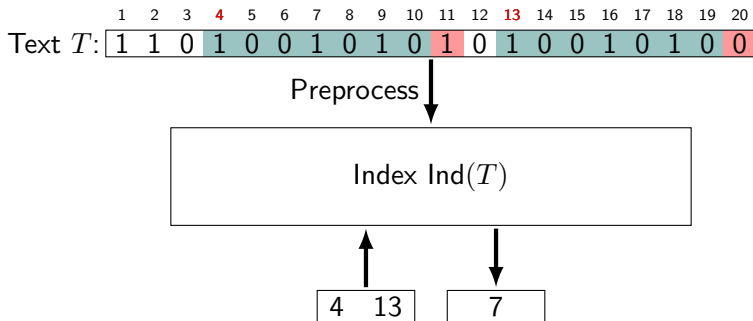
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



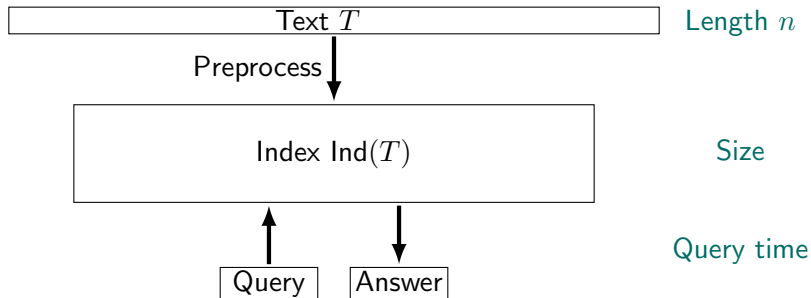
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



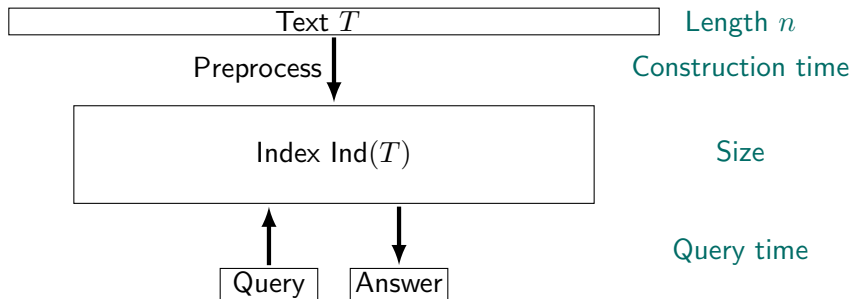
Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Text Indexing



Basic queries:

Random Access: Retrieve the character of T at a given position.

Pattern Matching: List the starting positions of the exact occurrences of a pattern.

Longest Common Extension: Find the max length of a string occurring at two given positions.

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

i	$SA[i]$	$T[SA[i]..n]$
1	20	0
2	19	00
3	14	0010100
4	5	0010101010010100
5	17	0100
6	12	010010100
7	3	010010101010010100
8	15	010100
9	10	01010010100
10	8	0101010010100
11	6	010101010010100
12	18	100
13	13	10010100
14	4	10010101010010100
15	16	10100
16	11	1010010100
17	2	1010010101010010100
18	9	101010010100
19	7	10101010010100
20	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

i	$SA[i]$	$T[SA[i]..n]$
1	20	0
2	19	00
3	14	0010100
4	5	0010101010010100
5	17	0100
6	12	010010100
7	3	010010101010010100
8	15	010100
9	10	01010010100
10	8	0101010010100
11	6	010101010010100
12	18	100
13	13	10010100
14	4	10010101010010100
15	16	10100
16	11	1010010100
17	2	1010010101010010100
18	9	101010010100
19	7	10101010010100
20	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

i	$SA[i]$	$T[SA[i]..n]$
1	20	0
2	19	00
3	14	0010100
4	5	0010101010010100
5	17	0100
6	12	010010100
7	3	010010101010010100
8	15	010100
9	10	01010010100
10	8	0101010010100
11	6	010101010010100
12	18	100
13	13	10010100
14	4	10010101010010100
15	16	10100
16	11	1010010100
17	2	1010010101010010100
18	9	101010010100
19	7	10101010010100
20	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,

i	$SA[i]$	$T[SA[i]..n]$
1	20	0
2	19	00
3	14	0010100
4	5	0010101010010100
5	17	0100
6	12	010010100
7	3	010010101010010100
8	15	010100
9	10	01010010100
10	8	0101010010100
11	6	010101010010100
12	18	100
13	13	10010100
14	4	10010101010010100
15	16	10100
16	11	1010010100
17	2	1010010101010010100
18	9	101010010100
19	7	10101010010100
20	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$SA[i]$	$T[SA[i]..n]$
1	20	0
2	19	00
3	14	0010100
4	5	0010101010010100
5	17	0100
6	12	010010100
7	3	010010101010010100
8	15	010100
9	10	01010010100
10	8	0101010010100
11	6	010101010010100
12	18	100
13	13	10010100
14	4	10010101010010100
15	16	10100
16	11	1010010100
17	2	1010010101010010100
18	9	101010010100
19	7	10101010010100
20	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Suffix Array

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	0
SA^{-1}	20	17	7	14	4	11	19	10	18	9	16	6	13	3	8	15	5	12	2	1

Suffix array

Manber & Myers, 1990

The suffix array $SA[1..n]$ of a text $T[1..n]$ is a permutation of $[1..n]$ such that $T[SA[i]..n]$ is the i th lexicographically smallest suffix of T .

Textbook solution for text indexing:

- Pattern Matching queries,
- Longest Common Extension queries,

Many **further applications** in string algorithms.

i	$LCP[i]$	$SA[i]$	$T[SA[i]..n]$
1	-	20	0
2	1	19	00
3	2	14	0010100
4	6	5	0010101010010100
5	1	17	0100
6	4	12	010010100
7	8	3	010010101010010100
8	3	15	010100
9	6	10	01010010100
10	5	8	0101010010100
11	7	6	010101010010100
12	0	18	100
13	3	13	10010100
14	7	4	10010101010010100
15	2	16	10100
16	5	11	1010010100
17	9	2	1010010101010010100
18	4	9	101010010100
19	6	7	10101010010100
20	1	1	11010010101010010100

Compact Indexes

Is the suffix array all we need?

Compact Indexes

Complexity

length: n

alphabet size: σ

text size: $n \cdot \lceil \log \sigma \rceil$ bits

SA size: $n \cdot \lceil \log n \rceil$ bits

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

**If the index does not fit the main memory,
processing times increase ≈ 100 times.**

Compact indexes: Can we implement suffix array functionality in $\mathcal{O}(n \log \sigma)$ bits?

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

**If the index does not fit the main memory,
processing times increase ≈ 100 times.**

Compact indexes: Can we implement suffix array functionality in $\mathcal{O}(n \log \sigma)$ bits?

Yes! Query time $\mathcal{O}(\log_{\sigma}^{\epsilon} n)$ to retrieve any entry $\text{SA}[i]$ (SA queries).

- CSA [Grossi & Vitter; STOC'00; SICOMP'05], FM index [Ferragina & Manzini; FOCS'00; JACM'05].

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

**If the index does not fit the main memory,
processing times increase ≈ 100 times.**

Compact indexes: Can we implement suffix array functionality in $\mathcal{O}(n \log \sigma)$ bits?

Yes! Query time $\mathcal{O}(\log_{\sigma}^{\epsilon} n)$ to retrieve any entry $\text{SA}[i]$ (SA queries).

- CSA [Grossi & Vitter; STOC'00; SICOMP'05], FM index [Ferragina & Manzini; FOCS'00; JACM'05].

Widely used in space-critical applications, incl. bioinformatics. [ACM Theory & Practice Award'22]

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

**If the index does not fit the main memory,
processing times increase ≈ 100 times.**

Compact indexes: Can we implement suffix array functionality in $\mathcal{O}(n \log \sigma)$ bits?

Yes! Query time $\mathcal{O}(\log_{\sigma}^{\epsilon} n)$ to retrieve any entry $\text{SA}[i]$ (SA queries).

- CSA [Grossi & Vitter; STOC'00; SICOMP'05], FM index [Ferragina & Manzini; FOCS'00; JACM'05].

Widely used in space-critical applications, incl. bioinformatics. [ACM Theory & Practice Award'22]

Many follow-ups and extensions, including suffix tree functionality [Sadakane; SODA'02; ToCS'07]

Compact Indexes

Complexity & Example (single human genome, two sets of chromosomes)

length: $n \approx 6.4 \cdot 10^9$,

alphabet size: $\sigma = 4$,

text size: $n \cdot \lceil \log \sigma \rceil$ bits ≈ 1.6 GB,

SA size: $n \cdot \lceil \log n \rceil$ bits ≈ 25.6 GB.

**If the index does not fit the main memory,
processing times increase ≈ 100 times.**

Compact indexes: Can we implement suffix array functionality in $\mathcal{O}(n \log \sigma)$ bits?

Yes! Query time $\mathcal{O}(\log^\epsilon n)$ to retrieve any entry $\text{SA}[i]$ (SA queries).

- CSA [Grossi & Vitter; STOC'00; SICOMP'05], FM index [Ferragina & Manzini; FOCS'00; JACM'05].

Widely used in space-critical applications, incl. bioinformatics. [ACM Theory & Practice Award'22]

Many follow-ups and extensions, including suffix tree functionality [Sadakane; SODA'02; ToCS'07]

Very recent result: [Kempa, K; 2026⁺] Unconditional lower bound $\Omega(\frac{\log \log n}{\log \log \log n})$ for $\sigma = \mathcal{O}(1)$.

Compressed Indexes



Compressed Indexes



A computational challenge:

- Sequential datasets are petabytes in size, predicted to reach exabytes this decade.
- Current techniques are not sufficient to effectively store and search them.



A computational challenge:

- Sequential datasets are petabytes in size, predicted to reach exabytes this decade.
- Current techniques are not sufficient to effectively store and search them.
- **Hope:** many datasets are **highly compressible**.
 - Human genomes share over 99.5% characters;
 - 10 000 genomes can be compressed to 4 GB (≈ 2.5 uncompressed genomes).

[GDC2]

Compressed Indexes



A computational challenge:

- Sequential datasets are petabytes in size, predicted to reach exabytes this decade.
- Current techniques are not sufficient to effectively store and search them.
- **Hope:** many datasets are **highly compressible**.
 - Human genomes share over 99.5% characters;
 - 10 000 genomes can be compressed to 4 GB (≈ 2.5 uncompressed genomes).
- Compression alone renders the text unreadable.

[GDC2]

Compressed Indexes



A computational challenge:

- Sequential datasets are petabytes in size, predicted to reach exabytes this decade.
- Current techniques are not sufficient to effectively store and search them.
- **Hope:** many datasets are **highly compressible**.
 - Human genomes share over 99.5% characters;
 - 10 000 genomes can be compressed to 4 GB (≈ 2.5 uncompressed genomes).
- Compression alone renders the text unreadable.

[GDC2]

Compressed indexes: Design indexes that simultaneously

- use space close to the compressed size of the text T , and
- efficiently answer various queries about T .

Compressibility Measures

Compressibility Measures

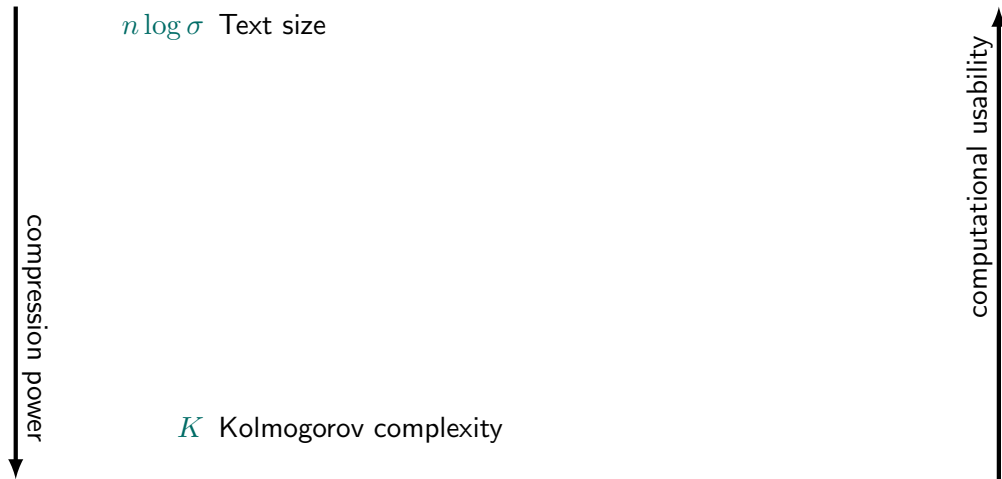
compression power



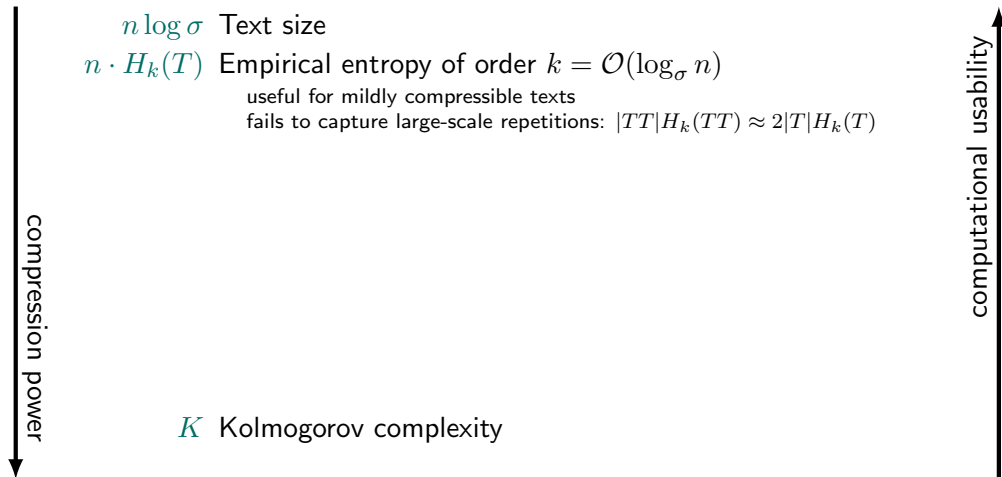
computational usability



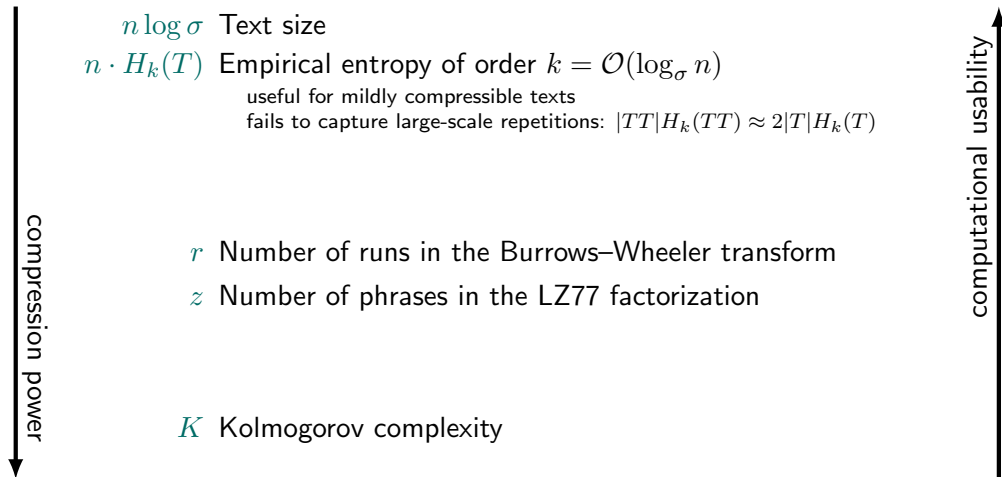
Compressibility Measures



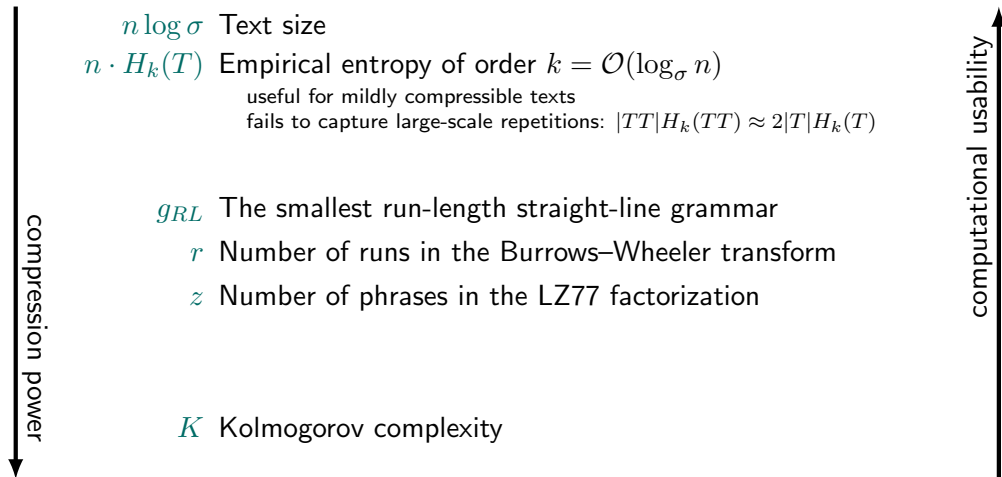
Compressibility Measures



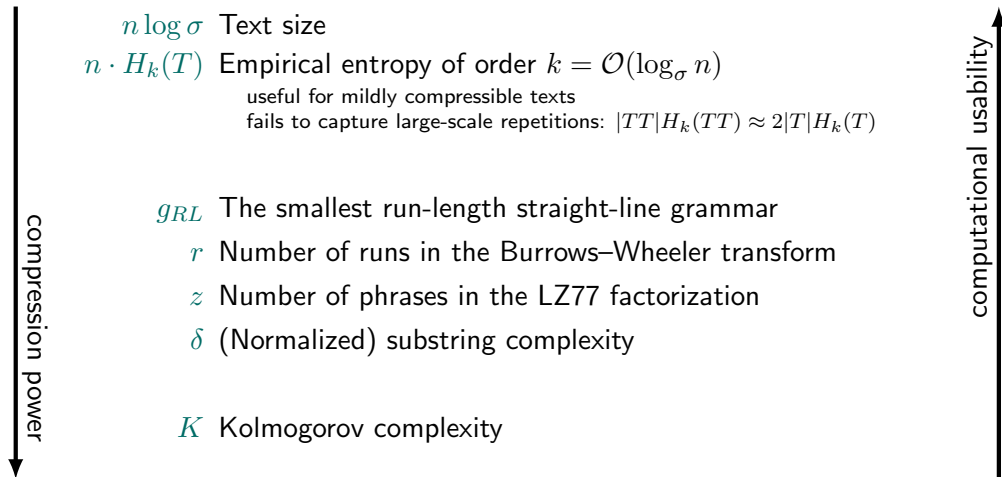
Compressibility Measures



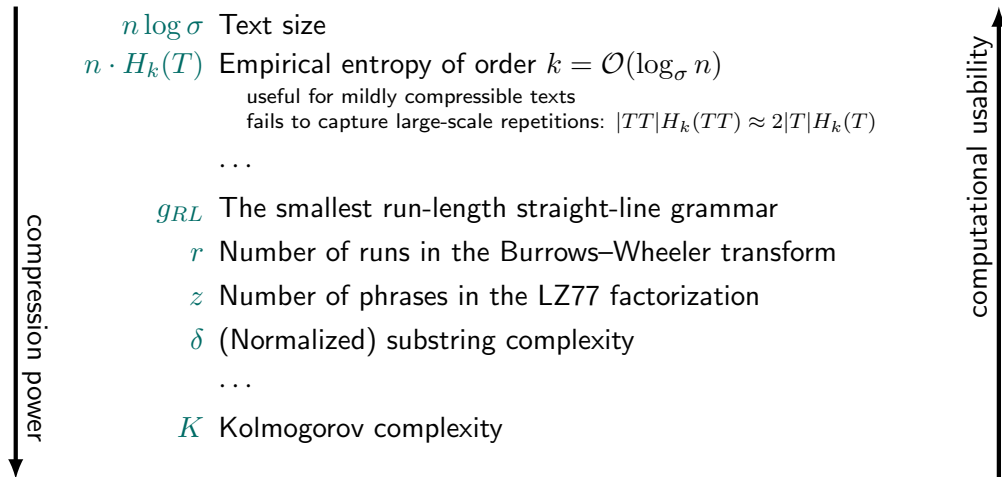
Compressibility Measures



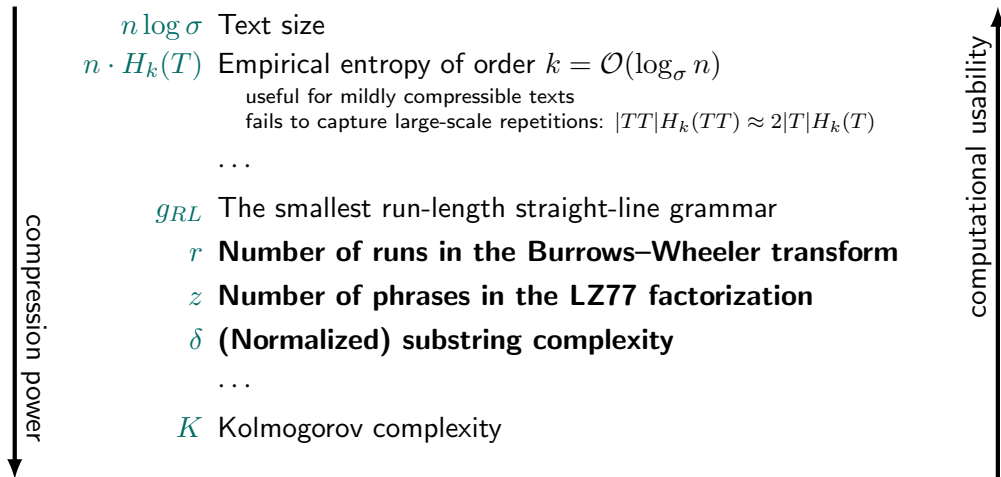
Compressibility Measures



Compressibility Measures



Compressibility Measures



Established Compressibility Measures

Lempel–Ziv Factorization (LZ77)

- Partition the text into single characters and phrases with previous occurrences.
- Encode each phrase by a pointer to an arbitrary previous occurrence.

|1|1|0|1 0|0 1 0|1 0 1 0 1 0|0 1 0 1 0 0| $z = 7$



Established Compressibility Measures

Lempel–Ziv Factorization (LZ77)

- Partition the text into single characters and phrases with previous occurrences.
- Encode each phrase by a pointer to an arbitrary previous occurrence.

|1|1|0|1 0|0 1 0|1 0 1 0 1 0|0 1 0 1 0 0| $z = 7$

Run-Length Burrows–Wheeler Transform

- Sort characters by the lexicographic order of their right contexts.
- Group runs of the same character.

i	$T[i-1]$	$T[i..n]$
1	\$	11010010101010010100
2	1	1010010101010010100
3	1	010010101010010100
4	0	10010101010010100
5	1	0010101010010100
6	0	010101010010100
7	0	10101010010100
8	1	0101010010100
9	0	101010010100
10	1	01010010100
11	0	1010010100
12	1	010010100
13	0	10010100
14	1	0010100
15	0	010100
16	0	10100
17	1	0100
18	0	100
19	1	00
20	0	0
21	0	

Established Compressibility Measures

Lempel–Ziv Factorization (LZ77)

- Partition the text into single characters and phrases with previous occurrences.
- Encode each phrase by a pointer to an arbitrary previous occurrence.

|1|1|0|1 0|0 1 0|1 0 1 0 1 0|0 1 0 1 0 0| $z = 7$

Run-Length Burrows–Wheeler Transform

- Sort characters by the lexicographic order of their right contexts.
- Group runs of the same character.

i	BWT[i]	$T[SA[i]..n]$
0	0	
1	0	0
2	1	00
3	1	0010100
4	1	0010101010010100
5	1	0100
6	1	010010100
7	1	010010101010010100
8	0	010100
9	1	01010010100
10	1	0101010010100
11	0	010101010010100
12	0	100
13	0	10010100
14	0	10010101010010100
15	0	10100
16	0	1010010100
17	1	1010010101010010100
18	0	101010010100
19	0	10101010010100
20	\$	11010010101010010100

Established Compressibility Measures

Lempel–Ziv Factorization (LZ77)

- Partition the text into single characters and phrases with previous occurrences.
- Encode each phrase by a pointer to an arbitrary previous occurrence.

|1|1|0|1 0|0 1 0|1 0 1 0 1 0|0 1 0 1 0 0| $z = 7$

Run-Length Burrows–Wheeler Transform

- Sort characters by the lexicographic order of their right contexts.
- Group runs of the same character.

$\text{rlbwt} = 0^2 \cdot 1^6 \cdot 0^1 \cdot 1^2 \cdot 0^6 \cdot 1^1 \cdot 0^2 \cdot \1

$r = 8$

i	BWT[i]	$T[\text{SA}[i] \dots n]$
0	0	
1	0	0
2	1	00
3	1	0010100
4	1	0010101010010100
5	1	0100
6	1	010010100
7	1	010010101010010100
8	0	010100
9	1	01010010100
10	1	0101010010100
11	0	010101010010100
12	0	100
13	0	10010100
14	0	10010101010010100
15	0	10100
16	0	1010010100
17	1	1010010101010010100
18	0	101010010100
19	0	10101010010100
20	\$	11010010101010010100

(Normalized) Substring Complexity δ

Substring Complexity

[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
[Christiansen, Ettiienne, K., Navarro, Prezza; ACM TALG'20]

For $\ell \in [1..n]$, let F_ℓ be the set of *distinct* length- ℓ substrings of T . Then,

$$\delta := \max_{\ell \in [1..n]} \frac{|F_\ell|}{\ell}.$$

(Normalized) Substring Complexity δ

Substring Complexity

[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
[Christiansen, Ettienne, K., Navarro, Prezza; ACM TALG'20]

For $\ell \in [1..n]$, let F_ℓ be the set of *distinct* length- ℓ substrings of T . Then,

$$\delta := \max_{\ell \in [1..n]} \frac{|F_\ell|}{\ell}.$$

Example:

$T = 11010010101010010100010101101110$

ℓ	$ F_\ell $	F_ℓ
1	2	0 1
2	4	00 01 10 11
3	8	000 001 010 011 100 101 110 111
4	12	0001 0010 0100 0101 0110 0111 1000 1001 1010 1011 1101 1110
5	16	00010 00101 01000 01001 01010 01011 01101 01110 10001 10010 10100 10101 10110 10111 11010 11011
$\vdots$	$\vdots$	$\ddots$

(Normalized) Substring Complexity δ

Substring Complexity

[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
[Christiansen, Ettienne, K., Navarro, Prezza; ACM TALG'20]

For $\ell \in [1..n]$, let F_ℓ be the set of *distinct* length- ℓ substrings of T . Then,

$$\delta := \max_{\ell \in [1..n]} \frac{|F_\ell|}{\ell}.$$

Example:

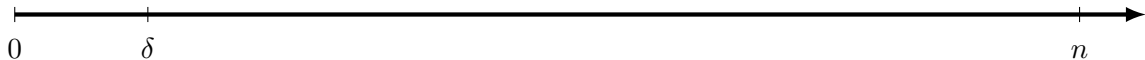
$T = 11010010101010010100010101101110$

ℓ	$ F_\ell $	F_ℓ
1	2	0 1
2	4	00 01 10 11
3	8	000 001 010 011 100 101 110 111
4	12	0001 0010 0100 0101 0110 0111 1000 1001 1010 1011 1101 1110
5	16	00010 00101 01000 01001 01010 01011 01101 01110 10001 10010 10100 10101 10110 10111 11010 11011
$\vdots$	$\vdots$	$\ddots$

$$\delta(T) = \max \left\{ \frac{2}{1}, \frac{4}{2}, \frac{8}{3}, \frac{12}{4}, \frac{16}{5}, \frac{18}{6}, \frac{21}{7}, \frac{23}{8}, \frac{23}{9}, \frac{23}{10}, \frac{22}{11}, \frac{21}{12}, \frac{20}{13}, \frac{19}{14}, \dots, \frac{1}{32} \right\} = \frac{16}{5}.$$

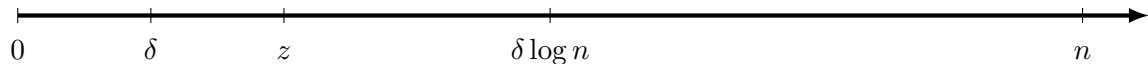
Substring Complexity vs Established Compressibility Measures

δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



Substring Complexity vs Established Compressibility Measures

δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT

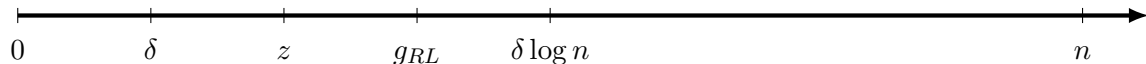


■ $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$

[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]

Substring Complexity vs Established Compressibility Measures

δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



■ $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$

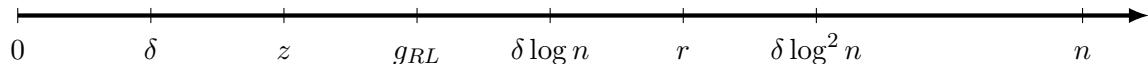
[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]

■ $\delta \leq g_{RL} \leq \mathcal{O}(\delta \log \frac{n}{\delta})$

[K., Navarro, Prezza; IEEE TIT'23]

Substring Complexity vs Established Compressibility Measures

δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



■ $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$

[Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]

■ $\delta \leq g_{RL} \leq \mathcal{O}(\delta \log \frac{n}{\delta})$

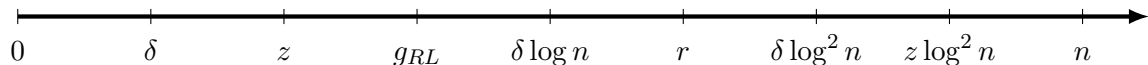
[K., Navarro, Prezza; IEEE TIT'23]

■ $\delta \leq r \leq \mathcal{O}(\delta \log \delta \log \frac{n}{\delta})$

[Kempa and Prezza; STOC'19], [Kempa and K.; FOCS'20]

Substring Complexity vs Established Compressibility Measures

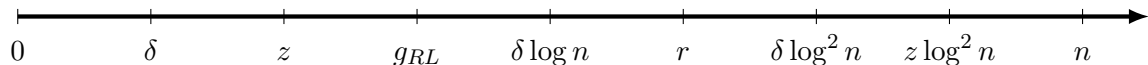
δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



- $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
- $\delta \leq g_{RL} \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [K., Navarro, Prezza; IEEE TIT'23]
- $\delta \leq r \leq \mathcal{O}(\delta \log \delta \log \frac{n}{\delta})$ [Kempa and Prezza; STOC'19], [Kempa and K.; FOCS'20]
- Substring complexity δ has been used to relate other measures:
 - $\Omega(z / \log \frac{n}{z}) \leq r \leq \mathcal{O}(z \log z \log \frac{n}{z})$ [Gagie, Navarro, Prezza; LATIN'18], [Kempa and K.; FOCS'20]

Substring Complexity vs Established Compressibility Measures

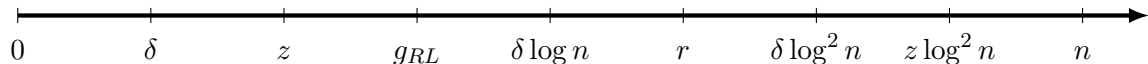
δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



- $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
- $\delta \leq g_{RL} \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [K., Navarro, Prezza; IEEE TIT'23]
- $\delta \leq r \leq \mathcal{O}(\delta \log \delta \log \frac{n}{\delta})$ [Kempa and Prezza; STOC'19], [Kempa and K.; FOCS'20]
- Substring complexity δ has been used to relate other measures:
 - $\Omega(z / \log \frac{n}{z}) \leq r \leq \mathcal{O}(z \log z \log \frac{n}{z})$ [Gagie, Navarro, Prezza; LATIN'18], [Kempa and K.; FOCS'20]
 - $r^{\text{rev}} \leq \mathcal{O}(r \log r \log \frac{n}{r})$ [Kempa and K.; FOCS'20], [Giuliani, Inenaga, Lipták, et al.; SOFSEM'21]

Substring Complexity vs Established Compressibility Measures

δ : substring complexity z : LZ77 factorization size g_{RL} : optimal RLSLP size r : number of runs in BWT



- $\delta \leq z \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [Raskhodnikova, Ron, Rubinfeld, Smith; RANDOM'07]
- $\delta \leq g_{RL} \leq \mathcal{O}(\delta \log \frac{n}{\delta})$ [K., Navarro, Prezza; IEEE TIT'23]
- $\delta \leq r \leq \mathcal{O}(\delta \log \delta \log \frac{n}{\delta})$ [Kempa and Prezza; STOC'19], [Kempa and K.; FOCS'20]
- Substring complexity δ has been used to relate other measures:
 - $\Omega(z / \log \frac{n}{z}) \leq r \leq \mathcal{O}(z \log z \log \frac{n}{z})$ [Gagie, Navarro, Prezza; LATIN'18], [Kempa and K.; FOCS'20]
 - $r^{\text{rev}} \leq \mathcal{O}(r \log r \log \frac{n}{r})$ [Kempa and K.; FOCS'20], [Giuliani, Inenaga, Lipták, et al.; SOFSEM'21]

Many further works on δ in string algorithms, combinatorics on words, and bioinformatics:

[AFI; Inf. Comput.'23], [CM; SPIRE'23], [BFGP; ISAAC'23], [BAL; iScience'24], [BCCKKP; DCC'24], ...

Compressed Indexes

Compressed Indexes: Selected Results

Query	Size	Reference

Remarks:

- Space is measured in machine words of $\Theta(\log n)$ bits each.
- Query times are $\log^{\mathcal{O}(1)} n$ each, except $(|P| + \text{occ}) \cdot \log^{\mathcal{O}(1)} n$ for pattern matching.

Compressed Indexes: Selected Results

Query	Size	Reference
-------	------	-----------

Random access	$\mathcal{O}(z \log \frac{n}{z})$	[Rytter; TCS'03], [Charikar et al.; SICOMP'05]
Pattern matching	$\mathcal{O}(z \log \frac{n}{z})$	[Claude and Navarro; MFCS'09], [Gagie et al.; LATIN'14], . . .
LCE queries	$\mathcal{O}(z \log \frac{n}{z})$	[I; CPM'17]

Remarks:

- Space is measured in machine words of $\Theta(\log n)$ bits each.
- Query times are $\log^{\mathcal{O}(1)} n$ each, except $(|P| + \text{occ}) \cdot \log^{\mathcal{O}(1)} n$ for pattern matching.

Compressed Indexes: Selected Results

Query	Size	Reference
-------	------	-----------

Random access	$\mathcal{O}(z \log \frac{n}{z})$	[Rytter; TCS'03], [Charikar et al.; SICOMP'05]
Pattern matching	$\mathcal{O}(z \log \frac{n}{z})$	[Claude and Navarro; MFCS'09], [Gagie et al.; LATIN'14], . . .
LCE queries	$\mathcal{O}(z \log \frac{n}{z})$	[I; CPM'17]
Pattern matching	$\mathcal{O}(r)$	[Mäkinen et al.; JCB'10], [Gagie, Navarro, Prezza; JACM'20]
SA queries	$\mathcal{O}(r \log \frac{n}{r})$	[Gagie, Navarro, Prezza; JACM'20]

Remarks:

- Space is measured in machine words of $\Theta(\log n)$ bits each.
- Query times are $\log^{\mathcal{O}(1)} n$ each, except $(|P| + \text{occ}) \cdot \log^{\mathcal{O}(1)} n$ for pattern matching.

Compressed Indexes: Selected Results

Query	Size	Reference
Random access	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[K., Navarro, Prezza; IEEE TIT'23]
Pattern matching	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[K., Navarro, Prezza; IEEE TIT'23], [K., Navarro, Olivares; Algorithmica'23]
LCE queries	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[Kempa and K.; FOCS'23]
SA queries	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[Kempa and K.; FOCS'23]
Random access	$\mathcal{O}(z \log \frac{n}{z})$	[Rytter; TCS'03], [Charikar et al.; SICOMP'05]
Pattern matching	$\mathcal{O}(z \log \frac{n}{z})$	[Claude and Navarro; MFCS'09], [Gagie et al.; LATIN'14], ...
LCE queries	$\mathcal{O}(z \log \frac{n}{z})$	[I; CPM'17]
Pattern matching	$\mathcal{O}(r)$	[Mäkinen et al.; JCB'10], [Gagie, Navarro, Prezza; JACM'20]
SA queries	$\mathcal{O}(r \log \frac{n}{r})$	[Gagie, Navarro, Prezza; JACM'20]

Remarks:

- Space is measured in machine words of $\Theta(\log n)$ bits each.
- Query times are $\log^{\mathcal{O}(1)} n$ each, except $(|P| + \text{occ}) \cdot \log^{\mathcal{O}(1)} n$ for pattern matching.

Compressed Indexes: Selected Results

Query	Size	Reference
Random access	$\mathcal{O}(g_{RL})$	[Bille et al.; SODA'11], [Ganardi et al.; FOCS'18], [Navarro et al.; SPIRE'22]
Pattern matching	$\mathcal{O}(g_{RL})$	[Christiansen et al., ACM TALG'20], [Navarro and Pacheco, CPM'25]
Random access	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[K., Navarro, Prezza; IEEE TIT'23]
Pattern matching	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[K., Navarro, Prezza; IEEE TIT'23], [K., Navarro, Olivares; Algorithmica'23]
LCE queries	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[Kempa and K.; FOCS'23]
SA queries	$\mathcal{O}(\delta \log \frac{n}{\delta})$	[Kempa and K.; FOCS'23]
Random access	$\mathcal{O}(z \log \frac{n}{z})$	[Rytter; TCS'03], [Charikar et al.; SICOMP'05]
Pattern matching	$\mathcal{O}(z \log \frac{n}{z})$	[Claude and Navarro; MFCS'09], [Gagie et al.; LATIN'14], ...
LCE queries	$\mathcal{O}(z \log \frac{n}{z})$	[I; CPM'17]
Pattern matching	$\mathcal{O}(r)$	[Mäkinen et al.; JCB'10], [Gagie, Navarro, Prezza; JACM'20]
SA queries	$\mathcal{O}(r \log \frac{n}{r})$	[Gagie, Navarro, Prezza; JACM'20]

Remarks:

- Space is measured in machine words of $\Theta(\log n)$ bits each.
- Query times are $\log^{\mathcal{O}(1)} n$ each, except $(|P| + \text{occ}) \cdot \log^{\mathcal{O}(1)} n$ for pattern matching.

Compressed Indexes in δ -Optimal Space

Theorem [KNP, IEEE TIT'23; KNO, Algorithmica'23; KK, FOCS'23]

Given the LZ77 parsing of a text $T \in [0 \dots \sigma]^n$ and any constant $\epsilon > 0$, we can in $\mathcal{O}(\delta \log^7 n)$ time construct a data structure of size $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ that answers:

- Random Access queries in $\mathcal{O}(\log n)$ time;
- LCE queries in $\mathcal{O}(\log n)$ time;
- Pattern Matching queries in $\mathcal{O}(|P| + (\text{occ} + 1) \log^\epsilon n)$ time;
- Suffix Array queries in $\mathcal{O}(\log^{4+\epsilon} n)$ time.

The construction algorithm is deterministic, and the running times are worst-case.

Compressed Indexes in δ -Optimal Space

Theorem [KNP, IEEE TIT'23; KNO, Algorithmica'23; KK, FOCS'23]

Given the LZ77 parsing of a text $T \in [0 \dots \sigma]^n$ and any constant $\epsilon > 0$, we can in $\mathcal{O}(\delta \log^7 n)$ time construct a data structure of size $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ that answers:

- Random Access queries in $\mathcal{O}(\log n)$ time;
- LCE queries in $\mathcal{O}(\log n)$ time;
- Pattern Matching queries in $\mathcal{O}(|P| + (\text{occ} + 1) \log^\epsilon n)$ time;
- Suffix Array queries in $\mathcal{O}(\log^{4+\epsilon} n)$ time.

The construction algorithm is deterministic, and the running times are worst-case.

Remark

$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ is the best possible size expressible as an increasing function of n, σ, δ .

Compressed Indexes in δ -Optimal Space

Theorem [KNP, IEEE TIT'23; KNO, Algorithmica'23; KK, FOCS'23]

Given the LZ77 parsing of a text $T \in [0.. \sigma)^n$ and any constant $\epsilon > 0$, we can in $\mathcal{O}(\delta \log^7 n)$ time construct a data structure of size $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ that answers:

- Random Access queries in $\mathcal{O}(\log n)$ time;
- LCE queries in $\mathcal{O}(\log n)$ time;
- Pattern Matching queries in $\mathcal{O}(|P| + (\text{occ} + 1) \log^\epsilon n)$ time;
- Suffix Array queries in $\mathcal{O}(\log^{4+\epsilon} n)$ time.

The construction algorithm is deterministic, and the running times are worst-case.

Remark

$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ is the best possible size expressible as an increasing function of n, σ, δ .
Smaller data structures cannot distinguish strings with different answers to queries.

Compressed Indexes in δ -Optimal Space

Theorem [KNP, IEEE TIT'23; KNO, Algorithmica'23; KK, FOCS'23]

Given the LZ77 parsing of a text $T \in [0 \dots \sigma]^n$ and any constant $\epsilon > 0$, we can in $\mathcal{O}(\delta \log^7 n)$ time construct a data structure of size $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ that answers:

- Random Access queries in $\mathcal{O}(\log n)$ time;
- LCE queries in $\mathcal{O}(\log n)$ time;
- **Pattern Matching** queries in $\mathcal{O}(|P| + (\text{occ} + 1) \log^\epsilon n)$ time;
- Suffix Array queries in $\mathcal{O}(\log^{4+\epsilon} n)$ time.

The construction algorithm is deterministic, and the running times are worst-case.

Remark

$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ is the best possible size expressible as an increasing function of n, σ, δ .
Smaller data structures cannot distinguish strings with different answers to queries.

Pattern Matching Queries

Pattern Matching: Length Reduction via Covers

Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m \dots 2m)$.

Pattern Matching: Length Reduction via Covers

Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m \dots 2m)$.

Idea: Shrink the text T while preserving the *leftmost* occurrences of supported patterns.

Pattern Matching: Length Reduction via Covers

Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m..2m)$.

Idea: Shrink the text T while preserving the *leftmost* occurrences of supported patterns.

ℓ -Cover

$C_\ell =$ the set of positions *covered* by the *leftmost occurrences* of length- ℓ substrings.

T : 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
1 1 0 1 0 0 1 0 1 0 1 0 1 0 0 1 0 1 0 0 0 1 0 1 0 1 1 0 1 1 1 0 $\ell = 3$

Pattern Matching: Length Reduction via Covers

Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m..2m)$.

Idea: Shrink the text T while preserving the *leftmost* occurrences of supported patterns.

ℓ -Cover

$C_\ell =$ the set of positions *covered* by the *leftmost occurrences* of length- ℓ substrings.

T : 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
1 1 0 1 0 0 1 0 1 0 1 0 1 0 0 1 0 1 0 0 0 1 0 1 0 1 1 0 1 1 1 0 $\ell = 3$

Lemma: The size of the ℓ -cover satisfies $|C_\ell| = \mathcal{O}(\ell \cdot \delta)$.

Pattern Matching: Length Reduction via Covers

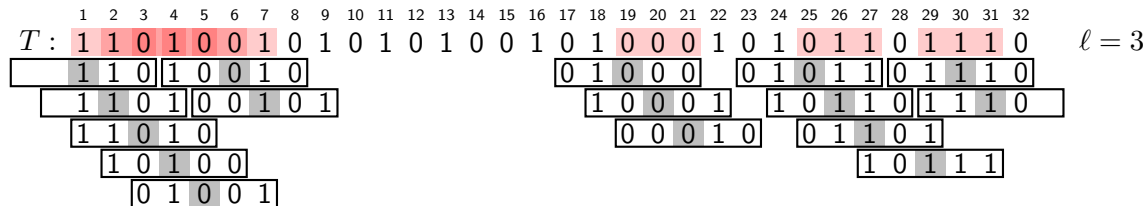
Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m \dots 2m)$.

Idea: Shrink the text T while preserving the *leftmost* occurrences of supported patterns.

ℓ -Cover

$C_\ell =$ the set of positions *covered* by the *leftmost occurrences* of length- ℓ substrings.



Lemma: The size of the ℓ -cover satisfies $|C_\ell| \leq |F_{2\ell-1}| + 2(\ell - 1) = \mathcal{O}(\ell \cdot \delta)$.

Pattern Matching: Length Reduction via Covers

Pattern Matching queries: Decision version

Decide if T contains an exact occurrence of a given pattern P of length $|P| \in [m \dots 2m)$.

Idea: Shrink the text T while preserving the *leftmost* occurrences of supported patterns.

ℓ -Cover

$C_\ell =$ the set of positions *covered* by the *leftmost occurrences* of length- ℓ substrings.

$T' :$ $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & \# & 0 & 0 & 0 & \# & 0 & 1 & 1 & \# & 1 & 1 & 1 \end{matrix}$ $\ell = 3$

Lemma: The size of the ℓ -cover satisfies $|C_\ell| \leq |F_{2\ell-1}| + 2(\ell - 1) = \mathcal{O}(\ell \cdot \delta)$.

Corollary: We can assume w.l.o.g. that $n \leq \mathcal{O}(m \cdot \delta)$ by removing positions not in C_{2m} .

Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.

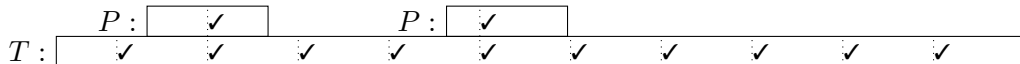
$T :$

✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
---	---	---	---	---	---	---	---	---	---

Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

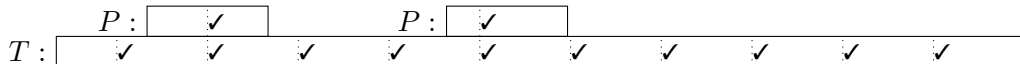
Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



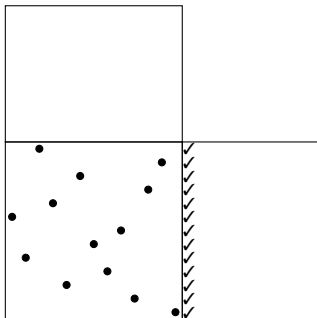
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



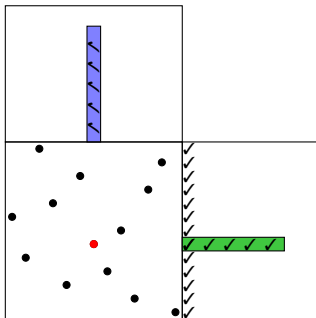
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



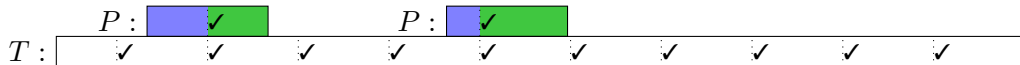
Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



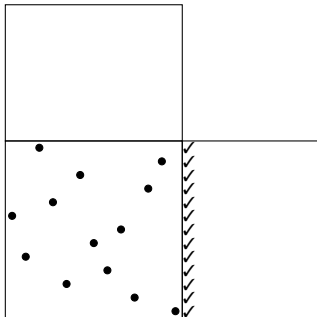
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.

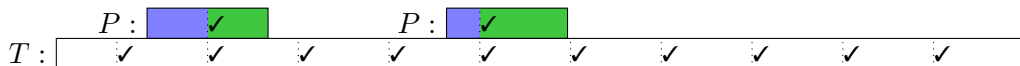


Query algorithm: For each partition $P = L \cdot R$:

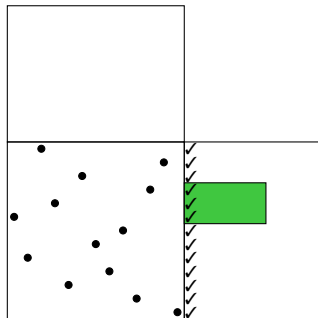
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



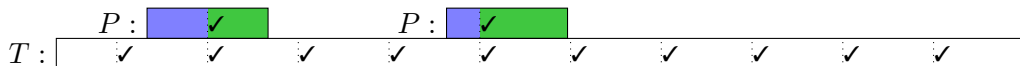
Query algorithm: For each partition $P = L \cdot R$:

- 1 Find anchors i s.t. R is a prefix of $T[i \dots n]$.
 - $\tilde{\mathcal{O}}(m)$ -time binary search like for the suffix array; random access needed.

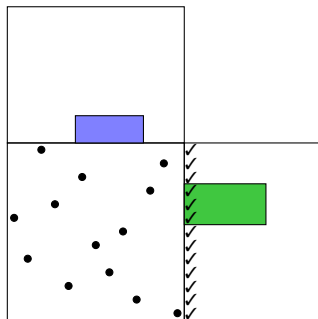
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



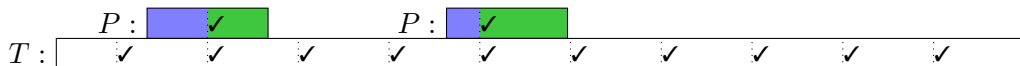
Query algorithm: For each partition $P = L \cdot R$:

- 1 Find anchors i s.t. R is a prefix of $T[i \dots n]$.
 - $\tilde{\mathcal{O}}(m)$ -time binary search like for the suffix array; random access needed.
- 2 Find anchors i s.t. L is a suffix of $T[1 \dots i]$.

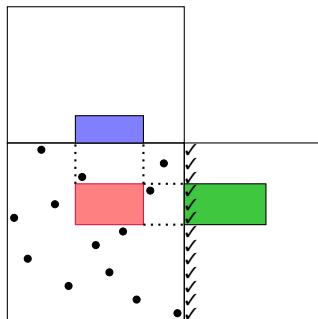
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



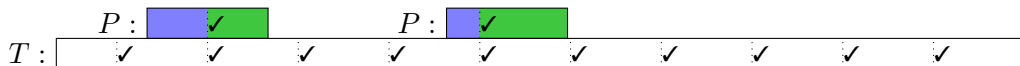
Query algorithm: For each partition $P = L \cdot R$:

- 1 Find anchors i s.t. R is a prefix of $T[i \dots n]$.
 - $\tilde{\mathcal{O}}(m)$ -time binary search like for the suffix array; random access needed.
- 2 Find anchors i s.t. L is a suffix of $T[1 \dots i]$.
- 3 Check for common positions (2D range emptiness).

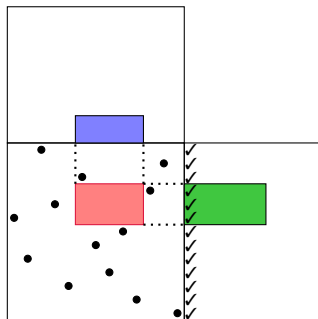
Pattern Matching: Orthogonal Range Queries

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Observation: Every occurrence of P covers a position in $m\mathbb{Z}$.



Data structure: The set of **anchors** $i \in m\mathbb{Z}$ sorted according to $T[i \dots n]$ and $\overline{T[1 \dots i]}$.



Query algorithm: For each partition $P = L \cdot R$:

- 1 Find anchors i s.t. R is a prefix of $T[i \dots n]$.
 - $\tilde{\mathcal{O}}(m)$ -time binary search like for the suffix array; random access needed.
- 2 Find anchors i s.t. L is a suffix of $T[1 \dots i]$.
- 3 Check for common positions (2D range emptiness).

Size: $\mathcal{O}(\delta)$ per range $[m \dots 2m)$ **Query time:** $\tilde{\mathcal{O}}(m^2)$

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

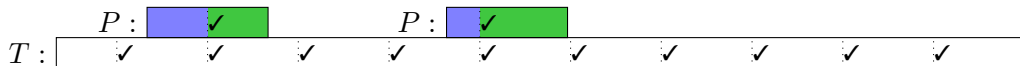
Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



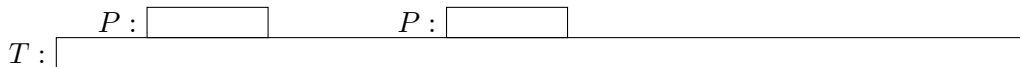
Idea: Choose S_m consistently so that one partition is valid for all occurrences of P .

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

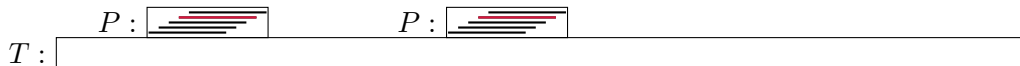
- 1 Assign uniformly random scores to all *distinct* substrings of length $2m/3$.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

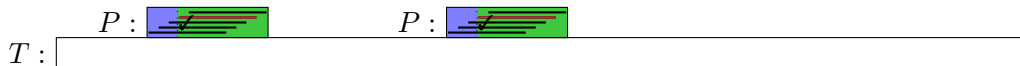
- 1 Assign uniformly random scores to all *distinct* substrings of length $2m/3$.
- 2 Partition at the starting positions of minimum-score substrings of the pattern.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

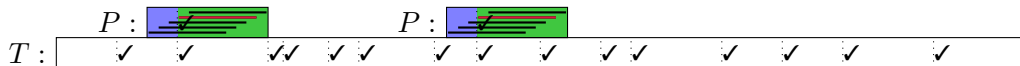
- 1 Assign uniformly random scores to all *distinct* substrings of length $2m/3$.
- 2 Partition at the starting positions of minimum-score substrings of the pattern.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

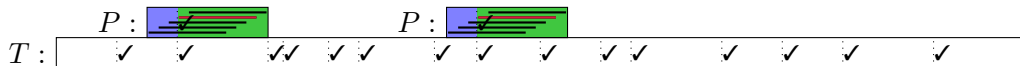
- 1 Assign uniformly random scores to all *distinct* substrings of length $2m/3$.
- 2 Partition at the starting positions of minimum-score substrings of the pattern.
- 3 Anchors: starting positions of minimum-score substrings within some length- m window.

Pattern Matching: Locally Consistent Anchors

Assumptions: $|P| \in [m \dots 2m)$ and $|T| = n \leq \mathcal{O}(m \cdot \delta)$

Bottleneck: $\Omega(m)$ partitions considered.

Goal: Replace $m\mathbb{Z}$ with a carefully chosen set S_m so that *one* partition suffices.



Idea: Choose S_m **consistently** so that one partition is valid for all occurrences of P .

Minimizers: a practical approach

[Roberts et al.; J. Comp. Bio.'04]

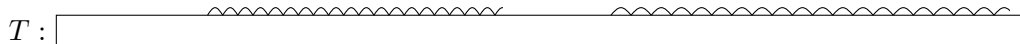
- 1 Assign uniformly random scores to all *distinct* substrings of length $2m/3$.
- 2 Partition at the starting positions of minimum-score substrings of the pattern.
- 3 Anchors: starting positions of minimum-score substrings within some length- m window.

Guarantee:

$\mathbb{E}[|S_m|] = \mathcal{O}(n/m) = \mathcal{O}(\delta)$ if every length- m window has $\Omega(m)$ length- $2m/3$ substrings.

Locally Consistent Anchors for Worst-Case Texts

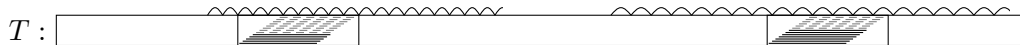
Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.



Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.



Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



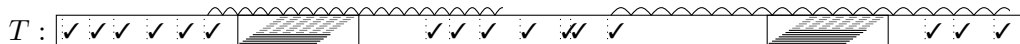
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



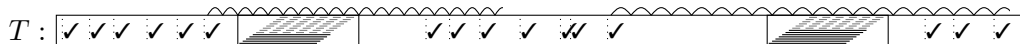
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



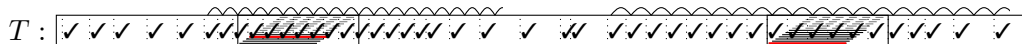
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



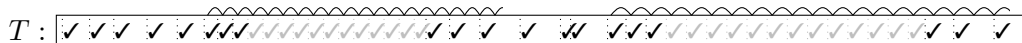
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.
- 3 Windows containing only substrings with periods below $m/6$ contribute many anchors...

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



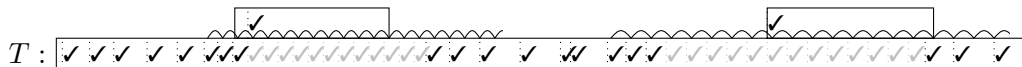
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.
- 3 Windows containing only substrings with periods below $m/6$ contribute many anchors...
 - but no **leftmost** occurrence is lost if we keep the leftmost two anchors for every run.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



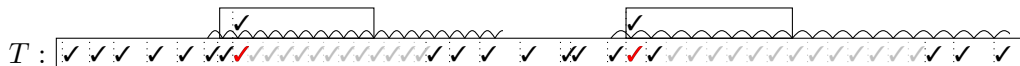
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.
- 3 Windows containing only substrings with periods below $m/6$ contribute many anchors...
 - but no **leftmost** occurrence is lost if we keep the leftmost two anchors for every run.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



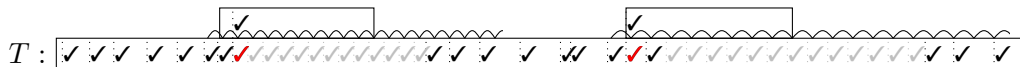
Solution:

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.
- 3 Windows containing only substrings with periods below $m/6$ contribute many anchors...
 - but no **leftmost** occurrence is lost if we keep the leftmost two anchors for every run.

Locally Consistent Anchors for Worst-Case Texts

Challenge: **Runs** in T of length $\geq 2m/3$ and period $p \leq m/6$.

- Windows contained in a period- p run have p distinct length- $2m/3$ substrings.
- Windows largely overlapping a period- p run also have few distinct length- $2m/3$ substrings.



Solution:

Guarantee: $\mathbb{E}[|S_m|] = \mathcal{O}(n/m)$ for **every** text T

- 1 When assigning scores to substrings, give preference to run boundaries, i.e., substrings whose period drops below $m/6$ upon removal of the first or last character.
 - Captures windows containing substrings with periods both above and below $m/6$.
 - $\mathcal{O}(n/m)$ anchors used: runs of length $\geq 2m/3$ and period $\leq m/6$ do not overlap too much.
- 2 Windows containing only substrings with periods above $m/6$ contribute few anchors:
 - Each window has $\geq m/6$ distinct substrings.
 - Adjacent windows share anchors with probability $\geq 1 - 6/m$.
- 3 Windows containing only substrings with periods below $m/6$ contribute many anchors...
 - but no **leftmost** occurrence is lost if we keep the leftmost two anchors for every run.

Efficient Deterministic Construction: String Synchronizing Sets

String Synchronizing Set

[Kempa, K.; STOC'19]

For $\tau \in [1 \dots n/2]$, τ -*synchronizing set* of a text T is a set $S \subseteq [1 \dots n]$ satisfying two properties:

consistency: if $T[i \dots i + 2\tau) = T[i' \dots i' + 2\tau)$, then $i \in S \iff i' \in S$;

density: $S \cap [i \dots i + \tau) \neq \emptyset \iff \text{period}(T[i \dots i + 3\tau - 1)) > \tau/3$.

Efficient Deterministic Construction: String Synchronizing Sets

String Synchronizing Set

[Kempa, K.; STOC'19]

For $\tau \in [1 \dots n/2]$, τ -*synchronizing set* of a text T is a set $S \subseteq [1 \dots n]$ satisfying two properties:

consistency: if $T[i \dots i + 2\tau) = T[i' \dots i' + 2\tau)$, then $i \in S \iff i' \in S$;

density: $S \cap [i \dots i + \tau) \neq \emptyset \iff \text{period}(T[i \dots i + 3\tau - 1)) > \tau/3$.

Theorem: Every $T \in \Sigma^n$ contains a τ -synchronizing set of size $\mathcal{O}(n/\tau)$.

Efficient Deterministic Construction: String Synchronizing Sets

String Synchronizing Set

[Kempa, K.; STOC'19]

For $\tau \in [1 \dots n/2]$, τ -synchronizing set of a text T is a set $S \subseteq [1 \dots n]$ satisfying two properties:

consistency: if $T[i \dots i + 2\tau) = T[i' \dots i' + 2\tau)$, then $i \in S \iff i' \in S$;

density: $S \cap [i \dots i + \tau) \neq \emptyset \iff \text{period}(T[i \dots i + 3\tau - 1)) > \tau/3$.

Theorem: Every $T \in \Sigma^n$ contains a τ -synchronizing set of size $\mathcal{O}(n/\tau)$.

Deterministic $\tilde{\mathcal{O}}(\delta + n/\tau)$ -time construction from the (restricted) recompression of T .

Efficient Deterministic Construction: String Synchronizing Sets

String Synchronizing Set

[Kempa, K.; STOC'19]

For $\tau \in [1 \dots n/2]$, τ -synchronizing set of a text T is a set $S \subseteq [1 \dots n]$ satisfying two properties:

consistency: if $T[i \dots i + 2\tau) = T[i' \dots i' + 2\tau)$, then $i \in S \iff i' \in S$;

density: $S \cap [i \dots i + \tau) \neq \emptyset \iff \text{period}(T[i \dots i + 3\tau - 1)) > \tau/3$.

Theorem: Every $T \in \Sigma^n$ contains a τ -synchronizing set of size $\mathcal{O}(n/\tau)$.

Deterministic $\tilde{\mathcal{O}}(\delta + n/\tau)$ -time construction from the (restricted) recompression of T .

Anchors for pattern matching queries:

- Elements of an $(m/3)$ -synchronizing set S .
- For each run of length $\geq m$ and period $\leq m/9$:
 - the leftmost two occurrences of a “canonical” rotation of the period (e.g., the Lyndon root).

Pattern Matching Queries: Selected Implementation Details

Data Structure:

$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ space

- 1 Restricted recompression of T (a locally consistent RLSLP).
- 2 For each “level” responsible for patterns of lengths in $[m \dots 2m)$:
 - a $m/3$ -synchronizing positions located within a $2m$ -cover, i.e., $S_{m/3} \cap C_{2m}$:
 - $\mathcal{O}(\delta)$ positions; extra positions needed for highly periodic patterns (“canonical” rotations)!
 - b Compacted tries for suffixes $T[i \dots n]$ and reversed prefixes $\overline{T[1 \dots i]}$ for $i \in S_{m/3} \cap C_{2m}$:
 - implemented using w -exponential search trees [Fischer, Gawrychowski; CPM’13].
 - c Orthogonal range emptiness structure [Chan, Larsen, Pătraşcu; SoCG’11] for the two orderings.

Pattern Matching Queries: Selected Implementation Details

Data Structure:

$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ space

- 1 Restricted recompression of T (a locally consistent RLSLP).
- 2 For each “level” responsible for patterns of lengths in $[m \dots 2m)$:
 - a $m/3$ -synchronizing positions located within a $2m$ -cover, i.e., $S_{m/3} \cap C_{2m}$:
 - $\mathcal{O}(\delta)$ positions; extra positions needed for highly periodic patterns (“canonical” rotations)!
 - b Compacted tries for suffixes $T[i \dots n]$ and reversed prefixes $\overline{T[1 \dots i]}$ for $i \in S_{m/3} \cap C_{2m}$:
 - implemented using w -exponential search trees [Fischer, Gawrychowski; CPM’13].
 - c Orthogonal range emptiness structure [Chan, Larsen, Pătraşcu; SoCG’11] for the two orderings.

Query Algorithm:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Partition $P = L \cdot R$.
 - a Parse P using restricted recompression (consistently with the parsing of T).
 - b Identify an $(m/3)$ -synchronizing position within P :
 - If $\text{period}(P) \leq m/9$, then, instead, identify a “canonical” rotation of the period of P .
- 2 Find R and $\overline{L}$ in the two tries.
- 3 Check if the underlying rectangle is non-empty.
 - Points in the rectangle are synchronizing positions preceded by L and followed by R .

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i + 2m)$ and $\overline{T[i - 2m..i)}$.

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i+2m)$ and $\overline{T[i-2m..i)}$.
- 3 If many $i \in S_{m/3}$ share the same contexts, store them using one weighted point.

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i+2m)$ and $\overline{T[i-2m..i]}$.
- 3 If many $i \in S_{m/3}$ share the same contexts, store them using one weighted point.
 - **Observation:** The leftmost representative of any point is within C_{4m} .
 - The number of distinct points is $|S_{m/3} \cap C_{4m}| = \mathcal{O}(\delta)$.

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i+2m)$ and $\overline{T[i-2m..i]}$.
- 3 If many $i \in S_{m/3}$ share the same contexts, store them using one weighted point.
 - **Observation:** The leftmost representative of any point is within C_{4m} .
 - The number of distinct points is $|S_{m/3} \cap C_{4m}| = \mathcal{O}(\delta)$.
- 4 Use weighted orthogonal range counting queries [Chazelle; SICOMP'88].

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i + 2m)$ and $\overline{T[i - 2m..i)}$.
- 3 If many $i \in S_{m/3}$ share the same contexts, store them using one weighted point.
 - **Observation:** The leftmost representative of any point is within C_{4m} .
 - The number of distinct points is $|S_{m/3} \cap C_{4m}| = \mathcal{O}(\delta)$.
- 4 Use weighted orthogonal range counting queries [Chazelle; SICOMP'88].
- 5 Specialized tools needed for periodic patterns.

Beyond the Decision Version of Pattern Matching Queries

Reporting Leftmost Occurrences:

$\mathcal{O}(m + \log^\epsilon \delta)$ time

- 1 Associate each point with the underlying position $i \in S_{m/3} \cap C_{2m}$.
- 2 Use orthogonal range minimum queries [Nekrich; SODA'21] instead of range emptiness.

Counting All Occurrences:

$\mathcal{O}(m + \log^{1+\epsilon} \delta)$ time

- 1 Store a point for every position $i \in S_{m/3}$ (not only positions within a cover!)
- 2 Trim the contexts of each $i \in S_{m/3}$ to $T[i..i+2m)$ and $\overline{T[i-2m..i]}$.
- 3 If many $i \in S_{m/3}$ share the same contexts, store them using one weighted point.
 - **Observation:** The leftmost representative of any point is within C_{4m} .
 - The number of distinct points is $|S_{m/3} \cap C_{4m}| = \mathcal{O}(\delta)$.
- 4 Use weighted orthogonal range counting queries [Chazelle; SICOMP'88].
- 5 Specialized tools needed for periodic patterns.

Remark: If the pattern is given as a fragment of T , the $\mathcal{O}(m)$ term reduces to $\mathcal{O}(\log n)$.

Conclusions and Future Directions

Summary

1 Text indexing:

- classic (suffix array)
- compact (FM index)
- **compressed (r -index, δ -SA)**

Summary

1 Text indexing:

- classic (suffix array)
- compact (FM index)
- **compressed (r -index, δ -SA)**

2 Tools:

- substring complexity δ
- string synchronizing sets
- two-dimensional orthogonal range queries
- the structure of highly aperiodic regions (runs) within the text

Summary

1 Text indexing:

- classic (suffix array)
- compact (FM index)
- **compressed (r -index, δ -SA)**

2 Tools:

- substring complexity δ
- string synchronizing sets
- two-dimensional orthogonal range queries
- the structure of highly aperiodic regions (runs) within the text

3 Results — $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ -space indexes:

- Random Access queries
- Longest Common Extension queries
- **Pattern Matching queries**
- Suffix Array queries

Open Questions

- 1 How much space is needed to answer fundamental queries *efficiently*?
in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

Open Questions

- 1 How much space is needed to answer fundamental queries *efficiently*?
in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching
- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [\[This talk\]](#)

Open Questions

- 1 How much space is needed to answer fundamental queries *efficiently*?
- in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching
- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]
 - $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]
- $\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

Open Questions

- 1 How much space is needed to answer fundamental queries *efficiently*?
- in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching
- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]
 - $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]
 - $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. $\mathcal{O}(z)$ or $\mathcal{O}(r)$ space? [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]
- $\mathcal{O}(z)$ space?

Open Questions

- 1 How much space is needed to answer fundamental queries *efficiently*?
in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching
- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]
 - $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]
 - $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. $\mathcal{O}(z)$ or $\mathcal{O}(r)$ space? [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]
 - $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. $\mathcal{O}(z)$ space? [Kempa, K.; FOCS'23]
 - $\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

■ $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

■ $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

■ $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

■ $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

■ $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

■ $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

■ $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

■ $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

■ For most queries $\mathcal{O}(\frac{\log n}{\log \log n})$ is sufficient and, as a function of n , necessary. [Kempa, K; SODA'26]

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

- $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

- $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

- For most queries $\mathcal{O}(\frac{\log n}{\log \log n})$ is sufficient and, as a function of n , necessary. [Kempa, K; SODA'26]

- What is the best possible query time for Pattern Matching? as a function of n , $|P|$, and occ

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

- $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

- $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

- For most queries $\mathcal{O}(\frac{\log n}{\log \log n})$ is sufficient and, as a function of n , necessary. [Kempa, K; SODA'26]

- What is the best possible query time for Pattern Matching? as a function of n , $|P|$, and occ

- What if we allow the query time to depend on δ ?

- $\mathcal{O}(\log \frac{n}{\delta})$ known for many queries [Gagie, Navarro, Prezza; JACM'20]

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

- $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

- $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

- For most queries $\mathcal{O}(\frac{\log n}{\log \log n})$ is sufficient and, as a function of n , necessary. [Kempa, K; SODA'26]

- What is the best possible query time for Pattern Matching? as a function of n , $|P|$, and occ

- What if we allow the query time to depend on δ ?

- $\mathcal{O}(\log \frac{n}{\delta})$ known for many queries [Gagie, Navarro, Prezza; JACM'20]

3 In general, what are the best-possible time-space tradeoffs?

Open Questions

1 How much space is needed to answer fundamental queries *efficiently*?

in $\log^{\mathcal{O}(1)} n$ time, $(|P| + \text{occ}) \log^{\mathcal{O}(1)} m$ time for Pattern Matching

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known and best-possible as an increasing function of n , σ , and δ . [This talk]

- $\mathcal{O}(g_{RL})$ known for Random Access. [Navarro, Olivares, Urbina; SPIRE'22]

$\mathcal{O}(z)$ or $\mathcal{O}(r)$ space?

- $\mathcal{O}(g_{RL})$ and $\mathcal{O}(r)$ known for Pattern Matching. [Navarro, Pacheco; CPM'25], [Gagie et al.; JACM'20]

$\mathcal{O}(z)$ space?

- $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ known for Suffix Array queries. [Kempa, K.; FOCS'23]

$\mathcal{O}(g_{RL})$ or $\mathcal{O}(r)$ space?

2 How fast can we answer queries in *compressed space*?

$\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$

- For most queries $\mathcal{O}(\frac{\log n}{\log \log n})$ is sufficient and, as a function of n , necessary. [Kempa, K; SODA'26]

- What is the best possible query time for Pattern Matching? as a function of n , $|P|$, and occ

- What if we allow the query time to depend on δ ?

- $\mathcal{O}(\log \frac{n}{\delta})$ known for many queries [Gagie, Navarro, Prezza; JACM'20]

3 In general, what are the best-possible time-space tradeoffs?

- Random Access is fully characterized for indexes of size $\Omega(\delta \log^{1+\epsilon} n)$ [Duyster, K.; ICALP'26]

Further Research Directions

1 Efficient construction from, say, LZ77 parsing:

- few indexes can be constructed efficiently;
- the $\log^{\mathcal{O}(1)} n$ factor is usually unoptimized, e.g., $\log^7 n$ for δ -SA

in $\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$ time:

[Kempa, K; FOCS'23]

Further Research Directions

- 1 Efficient construction from, say, LZ77 parsing: in $\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$ time:
 - few indexes can be constructed efficiently;
 - the $\log^{\mathcal{O}(1)} n$ factor is usually unoptimized, e.g., $\log^7 n$ for δ -SA [Kempa, K; FOCS'23]
- 2 Dynamic compressed indexing: maintain a compressed index as T changes over time.
■ Recent breakthroughs: [Nishimoto and Tabei; 2024⁺, 2026⁺]
 - $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ amortized expected space;
 - Updates in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected amortized time;
 - Pattern Matching in $\mathcal{O}(m \log^{\mathcal{O}(1)} n)$ expected time;
 - Suffix Array queries in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected time;

450+ pages.

Further Research Directions

- 1 Efficient construction from, say, LZ77 parsing: in $\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$ time:
 - few indexes can be constructed efficiently;
 - the $\log^{\mathcal{O}(1)} n$ factor is usually unoptimized, e.g., $\log^7 n$ for δ -SA [Kempa, K; FOCS'23]
- 2 Dynamic compressed indexing: maintain a compressed index as T changes over time.
 - Recent breakthroughs: [Nishimoto and Tabei; 2024⁺, 2026⁺]
 - $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ amortized expected space;
 - Updates in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected amortized time;
 - Pattern Matching in $\mathcal{O}(m \log^{\mathcal{O}(1)} n)$ expected time;
 - Suffix Array queries in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected time; 450+ pages.
 - What can be achieved deterministically or with high probability?
 - Can we get $\mathcal{O}(\log n)$ update and query time (matching the uncompressed state of the art) for basic queries like Random Access and LCE?

Further Research Directions

- 1 Efficient construction from, say, LZ77 parsing: in $\mathcal{O}(\delta \log^{\mathcal{O}(1)} n)$ time:
 - few indexes can be constructed efficiently;
 - the $\log^{\mathcal{O}(1)} n$ factor is usually unoptimized, e.g., $\log^7 n$ for δ -SA [Kempa, K; FOCS'23]
- 2 Dynamic compressed indexing: maintain a compressed index as T changes over time.
[Nishimoto and Tabei; 2024⁺, 2026⁺]
 - Recent breakthroughs:
 - $\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$ amortized expected space;
 - Updates in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected amortized time;
 - Pattern Matching in $\mathcal{O}(m \log^{\mathcal{O}(1)} n)$ expected time;
 - Suffix Array queries in $\mathcal{O}(\log^{\mathcal{O}(1)} n)$ expected time; 450+ pages.
 - What can be achieved deterministically or with high probability?
 - Can we get $\mathcal{O}(\log n)$ update and query time (matching the uncompressed state of the art) for basic queries like Random Access and LCE?
- 3 Compressibility measures:
 - δ is not *reachable*; we cannot store every string in $\mathcal{O}(\delta)$ space
 - Is there a reachable and efficiently computable lower bound of established measures?

Query	Time	Size	Reference
Random access	$\mathcal{O}(\log n)$	$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$	[KNP; IEEE TIT'23]
Pattern matching	$\mathcal{O}(m + \log^\epsilon \delta)$	$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$	[KNP; IEEE TIT'23][KNO; Algorithmica'23][K'24 ⁺]
LCE queries	$\mathcal{O}(\log n)$	$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$	[KK; FOCS'23]
SA queries	$\mathcal{O}(\log^{4+\epsilon} n)$	$\mathcal{O}(\delta \log \frac{n \log \sigma}{\delta \log n})$	[KK; FOCS'23]

Thank you for listening!